

Chapter 1

Chapter 1 (Main Ideas)

- **Abstraction**
- Performance (CPI)
- Amdahl's Law (limits on performance)
- Concurrency & Parallelism (methods to improve performance)

Abstraction in computer systems

Most of the time programming works because we don't need to know everything underneath.

CIS 20 is about looking underneath.

Applications

↑

High-level **languages** (C, Python)

↑

Assembly / ISA

↑

Microarchitecture

↑

Hardware (circuits, transistors)

What is abstraction?

- **Abstraction** is the technique of hiding unnecessary details while exposing only what is needed to use a system correctly.
 - hiding details
 - exposing an interface
 - separating “what” from “how”

Why abstraction exists ?

- **Without abstraction, every programmer would need to understand**
 - circuits
 - memory timing
 - instruction scheduling
 - cache coherence
- **Large systems would be impossible to build**

Abstraction in everyday terms

- **Car**
 - Interface: steering wheel, pedals
 - Hidden details: engine timing, fuel injection
- **Library reserve desk**
 - Interface: request book
 - Hidden details: shelving, cataloging
- **Elevator button**
 - Interface: press floor number
 - Hidden details: motors, control systems

Abstraction Is Powerful — But Not Free

- Abstraction lets us ignore hardware details
- We can write programs without knowing circuits or memory layout
- But abstraction **hides costs**
 - Time
 - Work
 - Resources
- Computer architecture studies **what abstraction hides**

“Abstraction lets us build systems”

Course Theme:

Abstraction Is Good But Don't Forget Reality

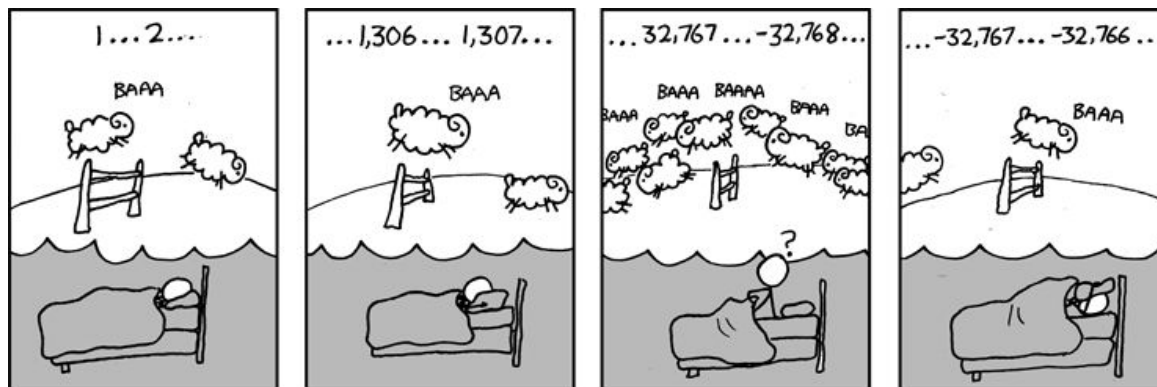
- **Most CS and CE courses emphasize abstraction**
 - Abstract data types
 - Asymptotic analysis
- **These abstractions have limits**
 - Especially in the presence of bugs
 - Need to understand details of underlying implementations
- **Useful outcomes from taking CIS20**
 - Become more effective programmers
 - Able to find and eliminate bugs efficiently
 - Able to understand and tune for program performance
 - Prepare for later “systems” classes in CS & ECE
 - Compilers, Operating Systems, Networks, Computer Architecture, Embedded Systems, Storage Systems, etc.

Great Reality #1:

Ints are not Integers, Floats are not Reals

■ Example 1: Is $x^2 \geq 0$?

■ Float's: Yes!



■ Int's:

- $40000 * 40000 \rightarrow 1600000000$
- $50000 * 50000 \rightarrow ??$

■ Example 2: Is $(x + y) + z = x + (y + z)$?

■ Unsigned & Signed Int's: Yes!

■ Float's:

- $(1e20 + -1e20) + 3.14 \rightarrow 3.14$
- $1e20 + (-1e20 + 3.14) \rightarrow ??$

Great Reality #1:

Ints are not Integers, Floats are not Reals

■ Example 1: Is $x^2 \geq 0$?

- $50000 * 50000 \rightarrow ??$

On a 32-bit signed **int**:

- Max value: 2,147,483,647
- $50000^2 = 2,500,000,000 \Rightarrow$ **X** overflow, Becomes **negative**
- $\text{int } x = 50000 \Rightarrow x^2 \geq 0$ is **FALSE** for ints.

■ Example 2: Is $(x + y) + z = x + (y + z)$?

▪ Float's:

- $1e20 + (-1e20 + 3.14) \rightarrow ??$

$(-1e20 + 3.14) \approx -1e20 \Rightarrow 3.14$ is too small to affect $1e20$ in floating point

$1e20 + (-1e20) = 01e20 + (-1e20) = 01e20 + (-1e20) = 0$

Final result: 0, not 3.14



Abstraction leaks

Computer Arithmetic

■ Does not generate random values

- Arithmetic operations have important mathematical properties

■ Cannot assume all “usual” mathematical properties

- Due to **finiteness** of representations
- Integer operations satisfy “ring” properties
 - Commutativity, associativity, distributivity
- Floating point operations satisfy “ordering” properties
 - Monotonicity, values of signs

■ Observation

- Need to understand which abstractions apply in which contexts
- Important issues for compiler writers and serious application programmers

Great Reality #2:

You've Got to Know Assembly

- **Chances are, you'll never write programs in assembly**
 - Compilers are much better & more patient than you are
- **But: Understanding assembly is key to machine-level execution model**
 - Behavior of programs in presence of bugs
 - High-level language models break down
 - Tuning program performance
 - Understand optimizations done / not done by the compiler
 - Understanding sources of program inefficiency
 - Implementing system software
 - Compiler has machine code as target
 - Operating systems must manage process state
 - Creating / fighting malware
 - x86 assembly is the language of choice!

Assembly is where
abstraction meets
reality.

Great Reality #3: Memory Matters

Random Access Memory Is an Unphysical Abstraction

■ Memory is not unbounded

- It must be allocated and managed
- Many applications are memory dominated

■ Memory referencing bugs especially pernicious

- Effects are distant in both time and space

■ Memory performance is not uniform

- Cache and virtual memory effects can greatly affect program performance
- Adapting program to characteristics of memory system can lead to major speed improvements

Memory behavior often dominates correctness and performance, and the abstraction of uniform access is misleading.

Memory Referencing Bug Example

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;  
  
double fun(int i) {  
    volatile struct_t s;  
    s.d = 3.14;  
    s.a[i] = 1073741824; /* Possibly out of bounds */  
    return s.d;  
}
```

```
fun(0)    → 3.14  
fun(1)    → 3.14  
fun(2)    → 3.13999998664856  
fun(3)    → 2.000000061035156  
fun(4)    → 3.14  
fun(6)    → Segmentation fault
```

- Result is system specific

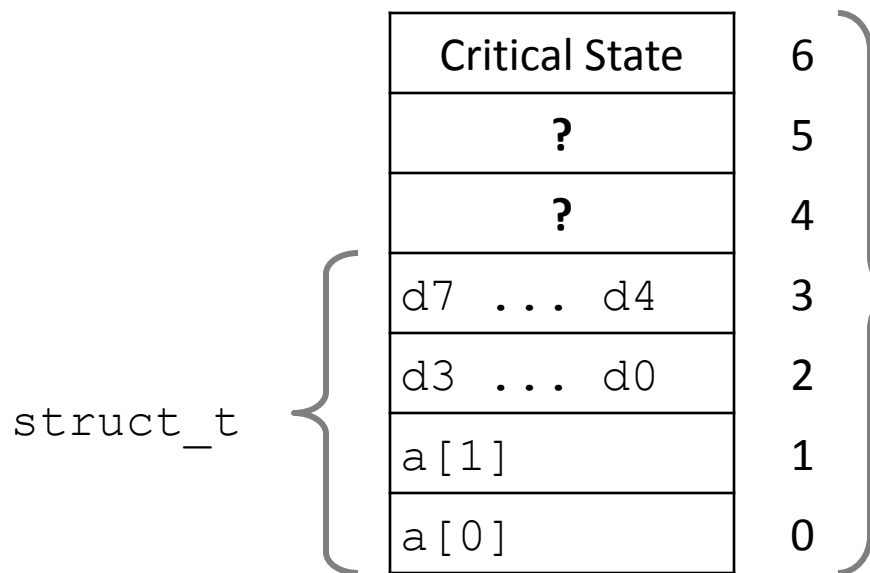
Memory Referencing Bug Example

```
typedef struct {
    int a[2];
    double d;
} struct_t;
```

```
fun(0)    → 3.14
fun(1)    → 3.14
fun(2)    → 3.1399998664856
fun(3)    → 2.00000061035156
fun(4)    → 3.14
fun(6)    → Segmentation fault
```

The bug exists — but it hides

Explanation:



```
double fun(int i) {
    volatile struct_t s;
    s.d = 3.14;
    s.a[i] = 1073741824; /* Possibly out of bounds */
    return s.d;
}
```

You overwrite the *low bits* of the double
 Floating-point representation is corrupted
 when $i \geq 2$

Memory Referencing Errors

- **C and C++ do not provide any memory protection**
 - Out of bounds array references
 - Invalid pointer values
 - Abuses of malloc/free
- **Can lead to nasty bugs**
 - Whether or not bug has any effect depends on system and compiler
 - Action at a distance
 - Corrupted object logically unrelated to one being accessed
 - Effect of bug may be first observed long after it is generated
- **How can I deal with this?**
 - Program in Java, Ruby, Python, ML, ...
 - Understand what possible interactions may occur
 - Use or develop tools to detect referencing errors (e.g. Valgrind)

Great Reality #4: There's more to performance than asymptotic complexity

- **Constant factors matter too!**
- **And even exact op count does not predict performance**
 - Easily see 10:1 performance range depending on how code written
 - Must optimize at multiple levels: algorithm, data representations, procedures, and loops
- **Must understand system to optimize performance**
 - How programs compiled and executed
 - How to measure program performance and identify bottlenecks
 - How to improve performance without destroying code modularity and generality

Memory System Performance Example

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}
```

4.3ms

2.0 GHz Intel Core i7 Haswell

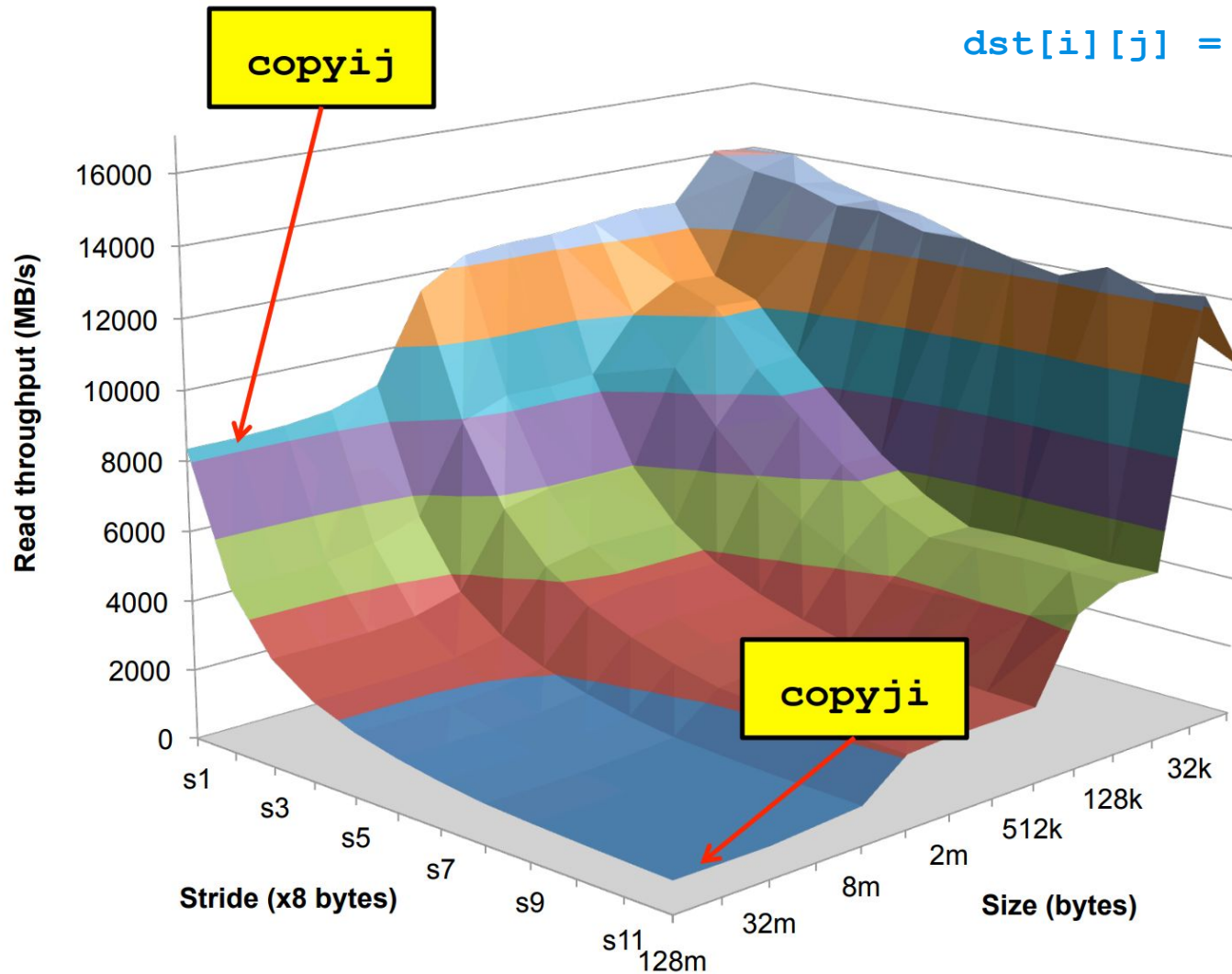
```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```

81.8ms

- Hierarchical memory organization, **not flat**
- Performance depends on access patterns
 - Including how step through multidimensional array

Why The Performance Differs

```
dst[i][j] = src[i][j]
```



Great Reality #5:

Computers do more than execute programs

- **They need to get data in and out**
 - I/O system critical to program reliability and performance

- **They communicate with each other over networks**
 - Many system-level issues arise in presence of network
 - Concurrent operations by autonomous processes
 - Coping with unreliable media
 - Cross platform compatibility
 - Complex performance issues

Recap of 5 Reality

Reality	Lesson
#1 Numbers	Representations aren't math
#2 Assembly	High-level models break
#3 Memory	Access patterns matter
#4 Performance	Big-O isn't enough
#5 I/O & Network	Programs live in the real world

Chapter 1 (Main Ideas)

- Abstraction
- **Performance (CPI)**
- Amdahl's Law (limits on performance)
- Concurrency & Parallelism (methods to improve performance)

Throughput vs. Response time

- **Response time:** How long does *one task* take?
- **Throughput:** How many tasks can we complete per unit time?
 - question: If I replace my CPU with a faster CPU, what improves?
 - question: If I add 16 processors to a server handling independent web searches, what improves?

Performance: CPI

- CPI: (clock) Cycles Per Instruction
- CPI: captures the average cost of abstraction
- CPI is a **measurement tool** that lets us reason about how abstraction behaves.
- CPI reveals what abstraction hides
- **CPI = average** number of clock cycles required to execute one instruction
- Lower CPI → better performance

$$\text{CPI} = \frac{\text{Total CPU cycles}}{\text{Total instructions}}$$

CPI and Abstraction

- CPI reveals what abstraction hides.

task lets programmers think in **instructions**, not cycles.

But task/abstraction hides:

- Pipeline depth
- Memory latency
- Branch penalties

Core Performance Formula

Term	Controlled by	Layer
Instruction Count	Algorithm, code	Software
CPI	Architecture + compiler	System
Clock Rate	Hardware	Physical

$$\text{CPU Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

Example

- CPU A: 2 GHz, CPI = 1

CPU B: 3 GHz, CPI = 2

Which executes more instructions per second?

- **A CPU cycle**, also called a **clock cycle** or **tick**, is the smallest increment of time in which a processor can perform a basic operation.
- **Clock Rate:**
 - 1 **Hz (Hertz)**: One cycle per second.
 - 1 **GHz (Gigahertz)**: One billion cycles per second.

2 GHz = 2 billions cycles/second

$$\text{CPI} = \frac{\text{Total CPU cycles}}{\text{Total instructions}}$$



$$\text{Instructions/sec} = \frac{\text{Clock Rate}}{\text{CPI}}$$

Instruction mix

- Different instruction types can have different CPIs.
 - Which type contributes the most cycles?

Type	Count	CPI
ALU	100	1
Load	50	4
Branch	20	2

$$CPI = \frac{\text{Total CPU cycles}}{\text{Total instructions}}$$



$$\text{Cycles} = \text{Instruction Count} \times CPI$$

Quiz

- Is the processor with the highest GHz necessarily fastest?
- Is the most common instruction necessarily consuming the most CPU time?

Chapter 1 (Main Ideas)

- Abstraction
- Performance (CPI)
- **Amdahl's Law (limits on performance)**
- Concurrency & Parallelism (methods to improve performance)

Amdahl's Law

■ What happens to total execution time if we speed up only one part of the system?

- faster hardware
- better algorithm
- parallel execution
- compiler optimization

$$T_{\text{new}} = (1 - \alpha)T_{\text{old}} + \frac{\alpha T_{\text{old}}}{k}$$

The textbook defines:

- T_{old}
→ Original execution time of the entire program
- α
→ Fraction of execution time spent in the part we improve
 - $0 \leq \alpha \leq 1$
 - Example: $\alpha = 0.6$ means *60% of the time*
- k
→ Speedup factor of the improved part
 - $k = 3$ means *3× faster*



Amdahl's Law

- Speedup is defined as:

$$S = \frac{T_{\text{old}}}{T_{\text{new}}}$$

Substitute the expression above:

$$S = \frac{T_{\text{old}}}{T_{\text{old}} \left[(1 - \alpha) + \frac{\alpha}{k} \right]}$$

Cancel T_{old} :

$$S = \frac{1}{(1 - \alpha) + \frac{\alpha}{k}}$$

Amdahl's Law

- Even if the improved part were infinitely fast, the unimproved part sets a **hard upper bound** on speedup.
- “Which part of the system actually matters?”
 - when k is very big:

$$S = \frac{1}{(1 - \alpha) + \alpha/k} \quad \Rightarrow \quad S_{\infty} = \frac{1}{(1 - \alpha)}$$

Example

Optimize the part that matters.
Which part of the system actually matters?

- Suppose a program takes 100 seconds.

90 seconds is for part A.

10 seconds is for part B.

$$S = \frac{T_{\text{old}}}{T_{\text{new}}}$$

- What happens if I make the 10-second part infinitely fast?"
- 100 sec $\rightarrow$ 90 sec
- Maximum speedup: $100/90=1.11 \Rightarrow 11\%$ faster

- If Part A becomes twice as fast:
- $90/2+10=55\text{s}$
- Speedup: $100/55 \approx 1.82 \Rightarrow 82\%$ faster

Chapter 1 (Main Ideas)

- Abstraction
- Performance (CPI)
- Amdahl's Law (limits on performance)
- **Concurrency & Parallelism (methods to improve performance)**

Concurrency

- **Concurrency** means a system has **multiple activities in progress** at the same time.
 - Does **not** require multiple CPUs
 - Activities may be interleaved

Examples:

- OS running multiple programs: printing and editing file
- A web server handling many connections
- Time-sharing on one core

Parallelism

■ **Parallelism** is using concurrency **to make the system run faster.**

■ “We want computers to do more, and we want them to run faster.”

- **Do more** → higher throughput
- **Run faster** → lower response time
- Concurrency helps both, but **parallelism is what reduces execution time.**

■ Important:

- Requires **actual simultaneous execution**
- Usually requires multiple cores, functional units, or hardware support
- *All parallelism is concurrency, but not all concurrency is parallelism.*

Parallelism exists at multiple levels

■ Level 1: Thread-Level (Task-Level) Parallelism (Highest Level)

- “This is parallelism at the program or system level.”
- Multiple **independent tasks or processes**
- Often managed by the OS
- Increase Throughput, but not necessarily response time

■ Level 2: Instruction-Level Parallelism (ILP)

- CPU overlaps or executes multiple **instructions** at once
- Happens **inside one program**: ex Pipelining, Superscalar execution, Out-of-order execution
- Improve: Lower CPI, Faster execution of a single thread

■ Level 3: Data-Level Parallelism (Lowest Level)

- Same operation applied to **many data items**: **One instruction, many pieces of data.**
- SIMD (Single Instruction, Multiple Data)
- improves Throughput for data-heavy workloads

How this connects to Amdahl's Law

Concept	Amdahl's Law Variable
Serial work	$1 - \alpha$
Parallel work	α
Degree of parallelism	k or N

- **Parallelism is the main tool for improving performance, but its benefits are limited by the amount of work that can be done concurrently.**
- **Amdahl's Law tells us the limit.**

Lab thinking questions?

- `a[i][j]`, What assumptions does this expression hide about how data is stored in memory?
- Parallelism always reduces runtime
- What is the difference between concurrency and parallelism?
- According to Amdahl's Law, why does speeding up a *small* part of a program often have little effect on total execution time?